

Парадигма визуального проектирования форм в Mozart ERP

Основные принципы

1. **Форма** — это метаданные (хранится в `meta.forms` и `meta.form_elements`).
2. **Поведение формы** — это **CALC-сущность** (экземпляр класса `CALC`, одна на всю форму).
3. Все методы и события формы хранятся в одной **CALC-сущности** в поле `ccode` (текст кода) или в файле по пути `cfilepath`.
4. **Идентификация метода/события** — по полному пути в иерархии контролов (`thisform.control.subcontrol.event`).

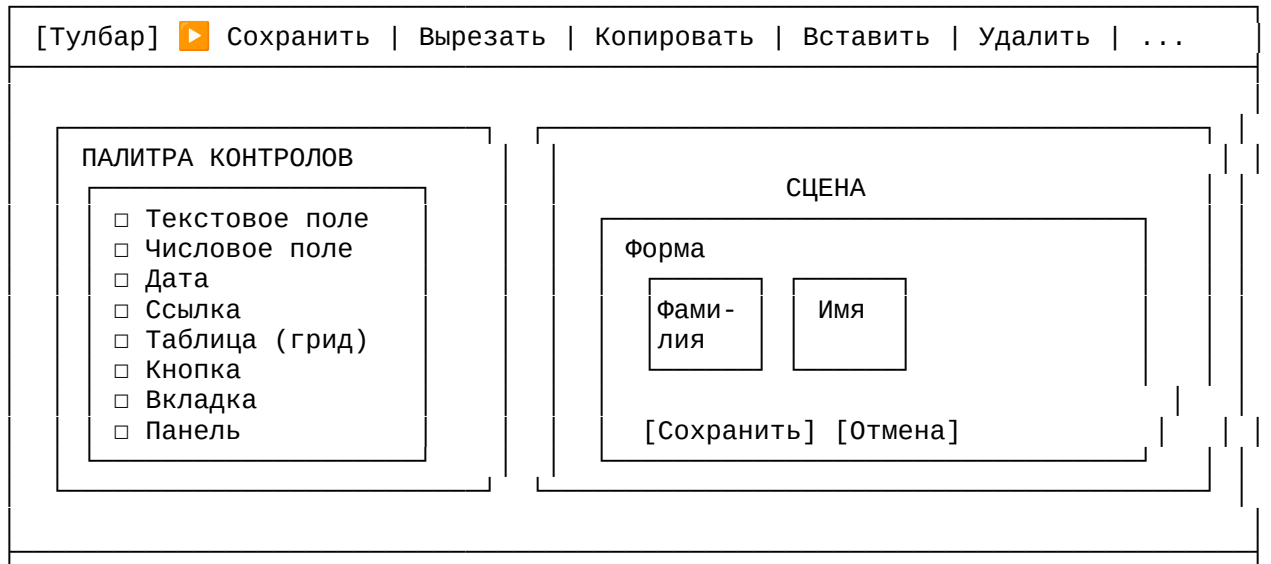
1. Архитектура RAD-среды в конфигураторе

1.1. Режимы работы с формой

Режим	Что делает	Когда
Просмотр	Показывает форму, контролы не редактируются	Для конечного пользователя
Редактирование (свойства)	Меняем свойства контролов, расположение	В конфигураторе
Визуальное проектирование	Перетаскиваем контролы, меняем размеры	В конфигураторе
Редактирование кода	Пишем обработчики (методы, события)	В конфигураторе

1.2. Окно визуального проектирования (аналог FoxPro 9.0)

text



ОКНО СВОЙСТВ (выбранного контрола)

Алиас: txt_lastname

Тип: TextBox

Подпись: Фамилия

Обязат.: ☒

readonly: ☐

Видимость: ☒

...

ОКНО МЕТОДОВ / СОБЫТИЙ (выбранного контрола)

События:

on_change : [▼] val_validate

on_click : [▼] show_message

[Редактировать код] → открывает редактор методов для этого события

2. Хранение кода методов и событий

2.1. Одна CALC-сущность на форму

Для каждой формы автоматически создаётся **один экземпляр сущности класса CALC**:

- **Алиас CALC-экземпляра:** `form_<form_alias>_logic`
- **Хранит** весь код методов и событий формы

Поля ext_calc (таблица расширения для сущности CALC):

Поле	Тип	Назначение
calias	VARCHAR(100)	Системный алиас (form_person_edit_logic)
sname	VARCHAR(500)	Человеческое название («Логика формы редактирования персоны»)
cfilepath	TEXT	Путь к файлу с кодом (опционально, для работы в IDE)
ccode	TEXT	Текст кода (если храним в БД)
chash	VARCHAR(64)	Хэш кода для контроля целостности
mcomment	TEXT	Описание
lisactive	BOOLEAN	Активна ли логика

2.2. Идентификация методов и событий

Внутри одной CALC-сущности методы и события идентифицируются по **полному пути в иерархии контролов**.

Формат идентификатора:

text

`thisform.<control_alias>.<event_name>`

Примеры:

Идентификатор

Что означает

<code>thisform.on_open</code>	Метод формы при открытии
<code>thisform.on_close</code>	Метод формы при закрытии
<code>thisform.on_save</code>	Метод формы при сохранении
<code>thisform.btn_save.on_click</code>	Событие <code>on_click</code> кнопки с алиасом <code>btn_save</code>
<code>thisform.grid_persons.on_dbl_click</code>	Событие <code>on_dbl_click</code> таблицы <code>grid_persons</code>
<code>thisform.txt_lastname.on_change</code>	Событие <code>on_change</code> текстового поля <code>txt_lastname</code>
<code>thisform.tab_main.subpanel.ref_city.on_select</code>	Событие контроля, вложенного в панель на вкладке

2.3. Структура кода в CALC-сущности

python

```
# /calc_scripts/form_person_edit_logic.py
# -*- coding: utf-8 -*-
# Логика формы редактирования персоны
# CALC алиас: form_person_edit_logic
# Версия: 1
# Создана: 2025-01-15

# ===== МЕТОДЫ ФОРМЫ =====

def on_open(context):
    """
    Вызывается при открытии формы.
    context = {
        'form': ссылка на форму,
        'params': переданные параметры,
        'app': ссылка на приложение,
        'db': сервис БД
    }
    """
    person_id = context['params'].get('person_id')
    if person_id:
        context['form'].load_data(person_id)

def on_close(context):
    """Вызывается при закрытии формы"""
    if context['form'].is_modified:
        return context['app'].confirm("Сохранить изменения?")
    return True

def on_save(context):
    """Сохранение формы"""
    if not context['form'].validate():
        context['app'].show_message("Проверьте заполнение полей", "warning")
        return False

    context['form'].save_to_db()
    context['app'].show_message("Сохранено", "info")
    return True

# ===== СОБЫТИЯ КОНТРОЛОВ =====
```

```

def thisform_btn_save_on_click(context):
    """Обработчик кнопки 'Сохранить'"""
    on_save(context)

def thisform_btn_cancel_on_click(context):
    """Обработчик кнопки 'Отмена'"""
    context['form'].close()

def thisform_txt_lastname_on_change(context):
    """При изменении фамилии — обновляем полное имя"""
    lastname = context['control'].value
    firstname = context['form'].get_control('txt_firstname').value
    if lastname and firstname:
        context['form'].get_control('txt_fullname').value = f"{lastname}{firstname}"

def thisform_grid_persons_on_dbl_click(context):
    """Двойной клик по строке таблицы"""
    selected = context['control'].get_selected()
    if selected:
        context['app'].open_form('person_edit',
        person_id=selected['instance_id'])

```

2.4. Хранение в БД

В `ext_calc` для записи с логикой формы:

Поле	Значение
<code>calias</code>	<code>form_person_edit_logic</code>
<code>sname</code>	Логика формы редактирования персоны
<code>ccode</code>	Текст кода (как выше)
<code>chash</code>	SHA-256 хэш кода
<code>lisactive</code>	<code>true</code>

2.5. Версионирование логики

CALC-сущность **версионизируется** как любая другая сущность Mozart:

- `data.entity_instances` — экземпляр логики
- `data.entity_versions` — версии логики
- `ext_calc` — данные конкретной версии

Что это даёт:

- Можно править логику, не ломая работающие формы
- Старая версия логики остаётся для старых периодов
- Аудит изменений

3. Жизненный цикл формы

Этап	Действие	Результат
1. Создание формы	Конфигуратор: создать запись в <code>meta.forms</code>	Пустая форма
2. Генерация CALC	Конфигуратор автоматически создаёт экземпляр CALC для формы	<code>form_<alias>_logic</code>
3. Визуальное проектирование	Перетаскивание контролов, настройка свойств	Заполняется <code>meta.form_elements</code>
4. Привязка событий	Для каждого события выбираем метод в CALC	В <code>mproperties_json</code> сохраняется идентификатор события
5. Редактирование кода	Открываем редактор кода, пишем логику	Код сохраняется в <code>ccode</code> CALC-экземпляра
6. Сохранение	Сохраняем форму и логику	Данные в БД
7. Запуск в L3	L3 загружает форму и логику, связывает события	Рабочая форма

4. Интеграция с CALC-менеджером

При открытии формы в L3:

1. **Загрузчик форм** читает `meta.forms` и `meta.form_elements`
2. **Строит визуальную часть** (создаёт контролы)
3. **Загружает CALC-логику** по алиасу `form_<form_alias>_logic`
4. **Привязывает события:** для каждого контрола находит в коде функцию с именем `thisform_<control_alias>_<event>` и подписывается
5. **Вызывает `on_open`** с переданными параметрами

Выполнение кода события:

python

```
# В L3, когда пользователь нажал кнопку
def on_button_click(self):
    calc_manager.execute_method(
        calc_alias='form_person_edit_logic',
        method_name='thisform_btn_save_on_click',
        context={'form': self, 'control': self.sender(), 'app': app, 'db': db}
    )
```

5. Преимущества подхода

Аспект	Преимущество
Единая CALC-сущность	Весь код формы в одном месте, легко версионировать
Идентификация по полному пути	Не нужно отдельной таблицы привязок
Версионирование	Можно откатить логику формы до любой версии
Права доступа	Стандартные права Mozart на выполнение CALC

Аудит

Хэш кода, история изменений

Разработка

Можно писать код во внешней IDE (по `cfilepath`)

6. Дополнительные возможности

6.1. Наследование форм

Форма может наследовать логику от другой формы:

- В `meta.forms` поле `parent_form_id`
- При загрузке L3 сначала загружает родительскую логику, затем переопределяет методы дочерней

6.2. Шаблоны форм

Для типовых сценариев:

- `OrdinaryDictionary` — форма для обычного справочника
- `OrdinaryDocument` — форма для документа
- `ReportForm` — форма для отчёта

Шаблон создаётся один раз, затем копируется под конкретную сущность.

6.3. Предпросмотр в конфигураторе

Возможность запустить форму в режиме просмотра прямо из конфигуратора:

- L3 не запускается
- Конфигуратор сам эмулирует выполнение (или запускает L3 в тестовом режиме)

6.4. Экспорт/импорт форм

Формы можно экспортировать в JSON и переносить между инсталляциями:

- Метаданные (`meta.forms`, `meta.form_elements`)
- Логика (`ext_calc` с кодом)

7. Итог

Компонент	Хранение	Назначение
Метаданные формы	<code>meta.forms</code> , <code>meta.form_elements</code>	Структура, расположение контролов, свойства
Логика формы	CALC-экземпляр (<code>form_<alias>_logic</code>)	Код методов и событий
Версионирование	<code>entity_instances</code> + <code>entity_versions</code>	История изменений логики
Идентификация событий	Полный путь (<code>thisform.control.event</code>)	Связь контроля с методом в коде
Никаких дополнительных таблиц. Всё в одной сущности CALC.		

